

OpenMP

OpenMP (`Open Multi-Processing`) — это стандарт, определяющий набор директив компилятора, библиотечных процедур и переменных среды окружения для создания многопоточных программ использующих общую память.

Стандарт предусматривает использование OpenMP для алгоритмических языков `C/C++` и `Fortran`.

Ссылки: <https://www.openmp.org>

Технология OpenMP — на данный момент одна из самых популярных технологий параллельного программирования для компьютеров с общей памятью. Стандарт был создан в 1997, однако его развитие и доработка происходит по сей день.

OpenMP реализует параллельные вычисления с помощью многопоточности, в которой ведущий (`master`) поток создаёт набор ведомых потоков, и задача распределяется между ними. Количество создаваемых потоков может регулироваться как самой программой при помощи вызова библиотечных процедур, так и извне, при помощи переменных окружения.

Цели OpenMP

Стандартизация:

- Единый стандарт для различных систем/архитектур/платформ с общей памятью.
- Определен совместными усилиями ведущих поставщиков программного и аппаратного обеспечения.

Краткость и выразительность:

- Небольшой набор директив и дополнительных опций (`clause`).

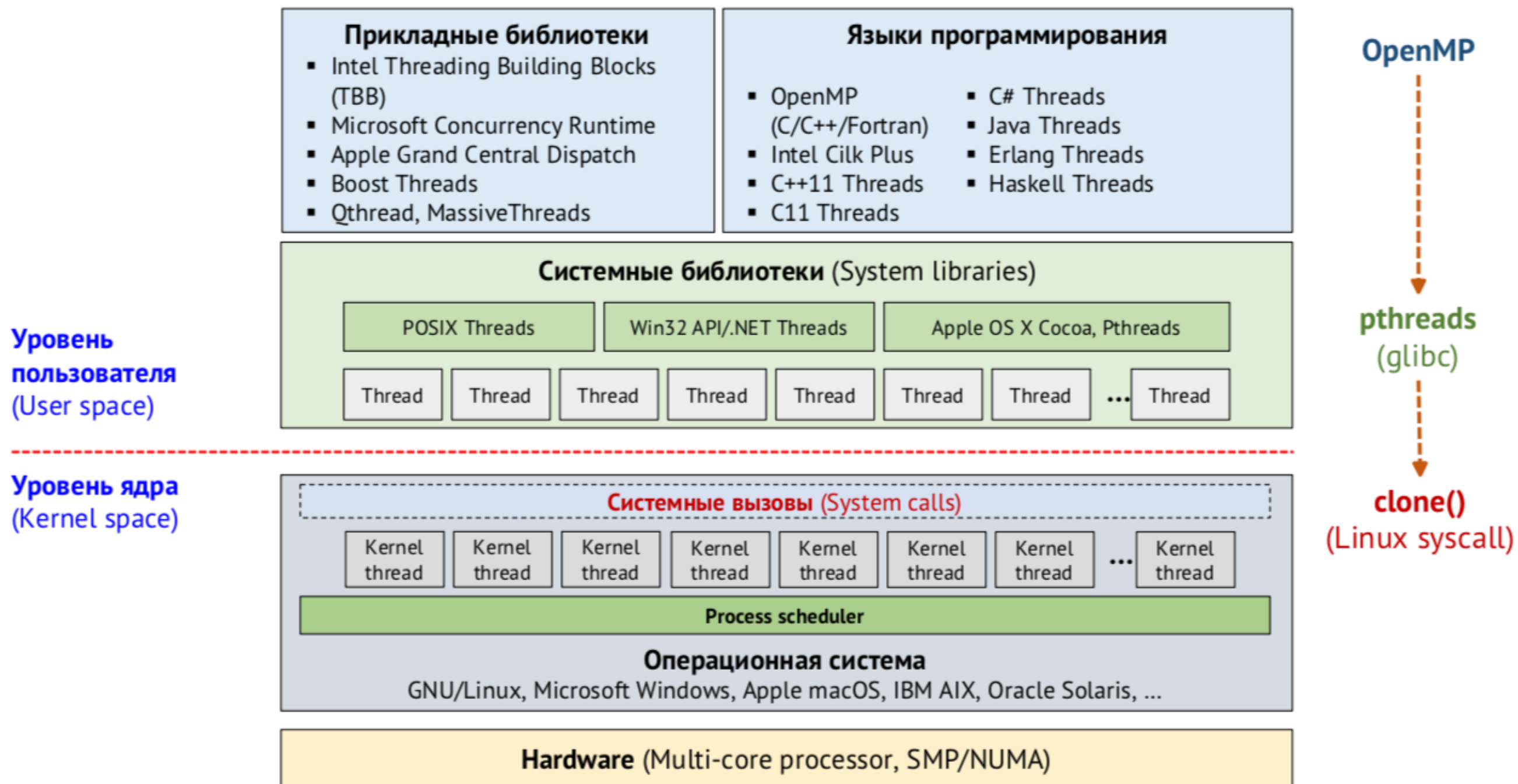
Простота использования:

- Упрощенный механизм создания параллельных программ, практически не влияющий на работу программиста.

Переносимость:

- Поддерживается в C/C++ и Fortran.
- Поддерживается на множестве платформ (`Unix/Linux`, `MacOS`, `Windows`).

Место OpenMP среди других методов распараллеливания



История OpenMP

- Начало разработки стандарта OpenMP, предназначалась для языка Fortran - 1997 год
- Intel, AMD*, ARM*, Cray*, IBM*, HP*, Micron*, NEC*, Nvidia*, Oracle* etc.
- Версии 1.0-2.5 (Октябрь 1997 – Май 2005)
- Первые версии, внедрение и развитие потокового распараллеливания циклов
- Версии 3.0, 3.1 (Май 2008 – Июль 2011)
- Добавление и развитие поддержки независимых задач
- Версия 4.0 (Июль 2013)
- Поддержка векторизации циклов (SIMD), поддержка ускорителей (target), зависимые задачи, встроенные механизмы обработки ошибок (cancel), пользовательские редукции, расширение атомарных конструкций
- Версия 4.5 (Ноябрь 2015), компиляторы GCC 6.0, Clang 3.8.
- Версия 5.1 (Ноябрь 2020), компиляторы GCC 11.0, Clang 12.0.
- Добавлена полная поддержка распараллеливания программ, написанных с использованием стандартов C11, C17, C++11, C++14, C++17 и C++20. Расширена поддержка специализированных аппаратных ускорителей.

Поддержка OpenMP компиляторами

— В GCC начиная с версии 4.2 реализована поддержка OpenMP для Си, C++ и Фортрана (на базе gfortran), а некоторые дистрибутивы (такие как Fedora Core 5) включили поддержку в GCC 4.1. В Clang и LLVM 3.7 поддерживается OpenMP 3.1.

— Intel C++ Compiler, Intel Fortran Compiler и Intel Parallel Studio поддерживают версию OpenMP 3.0, а также Intel Cluster OpenMP для программирования в системах с распределённой памятью.

Существуют также реализации в компиляторах IBM XL compiler, PGI (Portland group), Pathscale, HP Intel C++ Compiler, Intel Fortran Compiler и Intel Parallel Studio поддерживают версию OpenMP 3.0, а также Intel Cluster OpenMP для программирования в системах с распределённой памятью. Существуют также реализации в компиляторах IBM XL compiler, PGI (Portland group), Pathscale, HP.

Установка OpenMP

Установка OpenMP для `debian-based linux`:

```
sudo apt-get install libomp-dev
```

Чтобы собрать программу использующую OpenMP
следует указать компилятору флаг `-fopenmp`,
например:

```
gcc -fopenmp -o example example.c
```

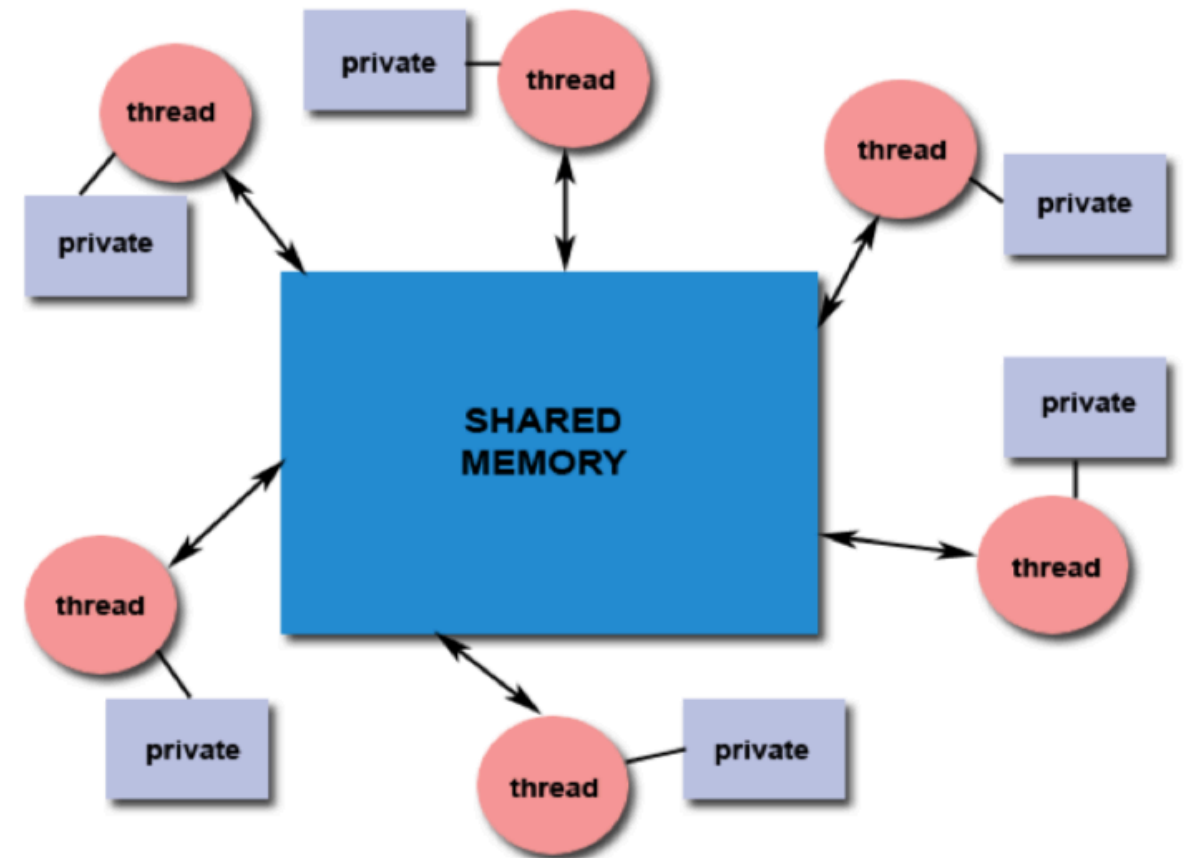

Версия стандарта

Узнать версию стандарта возможно с помощью макроса `_OPENMP`. Данный макрос содержит 6 цифр в формате `yyyy mm`, где первые 4 цифры это год, а две последующие это месяц создания формата.

```
int main(int argc, char** argv){  
  
    #ifdef _OPENMP  
        printf("OpenMP version is: %u.\n", _OPENMP);  
    #else  
        printf("OpenMP not exists.\n");  
    #endif  
  
    return 0;  
}
```


Модель с разделяемой памятью

- Все потоки имеют доступ к глобальной общей памяти.
- Данные могут быть общие и приватные.
- Общие данные доступны всем потокам.
- Приватные данные доступны только одному потоку-владельцу.
- Требуется синхронизация для доступа к общим данным.
- OpenMP использует подход SPMD (Single Program Multiply Data), то есть используется один и тот же код для описания поведения нескольких параллельных нитей.



Когда использовать OpenMP?

— Целевая платформа является многопроцессорной или многоядерной.

Если приложение полностью использует ресурсы одного ядра или процессора, то, сделав его многопоточным при помощи OpenMP, вы почти наверняка повысите его быстродействие.

— Приложение должно быть кроссплатформенным.

OpenMP – кроссплатформенный и широко поддерживаемый API. А так как он реализован на основе директив pragma, приложение можно скомпилировать даже при помощи компилятора, не поддерживающего стандарт OpenMP.

— Выполнение циклов можно распараллелить.

Весь свой потенциал OpenMP демонстрирует при организации параллельного выполнения циклов. Если в приложении есть длительные циклы без зависимостей, OpenMP – идеальное решение.

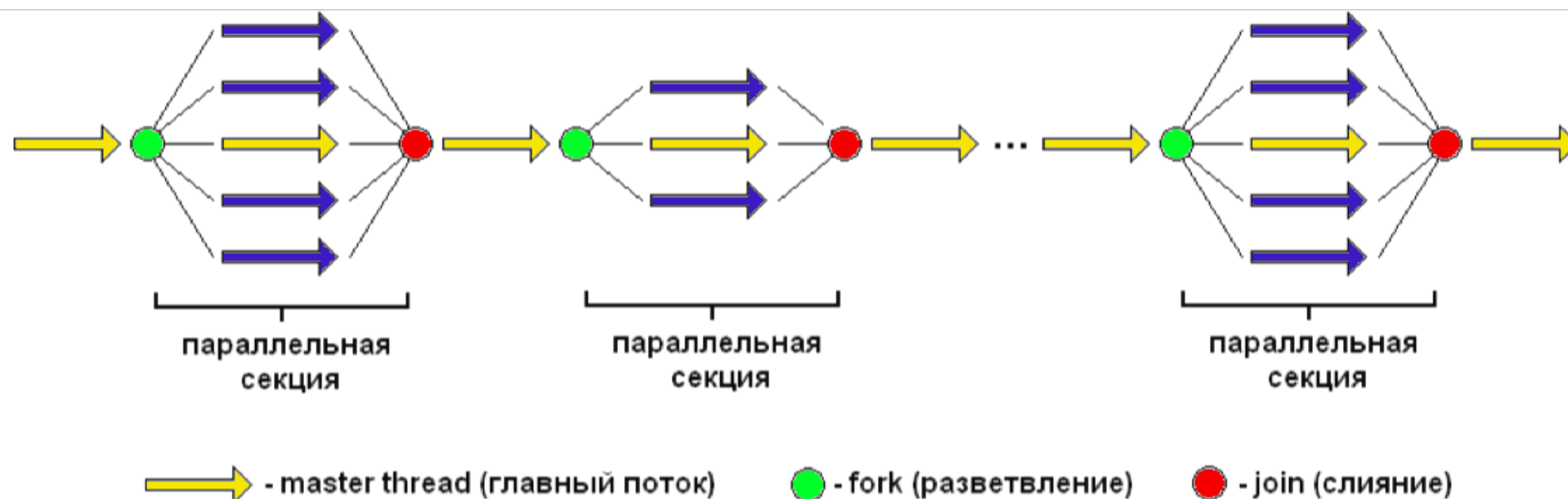
— Перед выпуском приложения нужно повысить его быстродействие.

Так как технология OpenMP не требует переработки архитектуры приложения, она прекрасно подходит для внесения в код небольших изменений, позволяющих повысить его быстродействие.

— Однако OpenMP – не панацея от всех бед.

Технология ориентирована в первую очередь на разработчиков высокопроизводительных вычислительных систем и наиболее эффективна, если код включает много циклов и работает с разделяемыми массивами данных.

Структура OpenMP программы



Программу написанную с помощью OpenMP чаще всего можно представить в виде набора последовательных блоков разветвления и слияний. Программа начинается с последовательной области, где работает одна нить, последовательно расширяясь в местах, где требуется многопоточность и сужаясь, там где многопоточность избыточна.

- Программа представляется в виде последовательных участков кода (`serial code`) и параллельных регионов (`parallel region`).
- Каждый поток имеет личный номер, для привязки к нему логики исполняемой параллельной программы.
- Мастер поток (`master`) имеет номер 0.
- Память процесса (`heap`) является общей для всех потоков.
- OpenMP реализует управление потоками, синхронизации, атомарные действия.

Модель параллельной программы

Последовательная область – один процесс (нить), вход в параллельную область – порождение некоторого числа процессов, их завершение, нить-мастер.

Параллельные области могут быть вложенными друг в друга.

В отличие от полноценных процессов, порождение нитей является относительно быстрой операцией, поэтому частые порождения и завершения нитей не так сильно влияют на время выполнения программы.

Для написания эффективной параллельной программы необходимо, чтобы все нити были равномерно загружены, что достигается тщательной балансировкой загрузки при помощи механизмов OpenMP.

Необходимость синхронизации доступа к общим данным – существенна. Само наличие данных, общих для нескольких нитей, приводит к конфликтам при одновременном несогласованном доступе.

Значительная часть функциональности OpenMP предназначена для осуществления различного рода синхронизаций работающих нитей.

OpenMP не выполняет синхронизацию доступа различных нитей к одним и тем же данным. Пользователь должен явно использовать директивы синхронизации или соответствующие библиотечные функции. При доступе каждой нити к своей области памяти никакая синхронизация не требуется.

Директивы OpenMP

Значительная часть функциональности OpenMP реализуется при помощи директив компилятору. Они должны быть явно применены пользователем. Директивы OpenMP в программах на языке C/C++ и являются указаниями препроцессору, начинающимися с `#pragma omp`. Формат директивы на C/C++:

`#pragma omp <имя директивы> [опция [[, ] опция ...]`

Программы строятся таким образом, чтобы если препроцессор не поддерживает OpenMP, программа всё равно запустилась бы в однопоточном режиме и наоборот.

Пример использования директивы:

```
#pragma omp parallel
{
    printf("parallel section\n");
}
```

В данном случае директива `parallel` применённая к блоку позволяет разделить исполнение кода внутри блока на некоторое количество потоков. Выделение пула потоков, синхронизация по их завершению генерируется препроцессором благодаря указанной прагме.

Далее за директивой может идти опция. Опция (`clause`) – это необязательный модификатор директивы, влияющий на её поведение. Списки опций, поддерживаемые каждой директивой, различаются, а пять директив (`master`, `critical`, `flush`, `ordered` и `atomic`) вообще не поддерживают опции.

Функции OpenMP

Кроме директив препроцессора OpenMP предоставляет также набор функций для управления состоянием программы.

Чтобы задействовать функции библиотеки OpenMP периода выполнения (исполняющей среды), в программу нужно включить заголовочный файл `omp.h`. Если в приложении используются только OpenMP директивы, включать этот файл не требуется.

Все функции, используемые в OpenMP, начинаются с префикса (приставки) `omp_`. Если пользователь не будет использовать в программе имён, начинающихся с такого префикса, то конфликтов с объектами OpenMP заведомо не будет. В языке C/C++, кроме того, является существенным регистр символов в названиях функций. Названия функций OpenMP записываются строчными буквами.

Некоторые функции:

Функция возвращает астрономическое время в секундах, прошедшее с некого момента в прошлом.

```
double omp_get_wtime(void);
```

Функция возвращает разрешение таймера с секундах (шаг).

```
double omp_get_wtick(void);
```

Функция возвращает номер текущей нити.

```
int omp_get_thread_num(void);
```

Функция возвращает количество запущенных в параллельной области нитей:

```
int omp_get_num_threads();
```

Функция возвращает кол-во доступных процессоров:

```
int omp_get_num_procs();
```

Функция позволяет установить кол-во нитей в пуле:

```
void omp_set_num_threads(int num);
```


Опции OpenMP

Опции в OpenMP указываются после директив. Например переменные в параллельных секциях можно разделить на общие (`shared`) и локальные (`private`) соответственно выставить область видимости переменных для той или иной секции позволяют одноимённые опции. По умолчанию все переменные, порожденные вне параллельной области при входе в неё являются общими.

```
int thrd;
```

```
#pragma omp parallel private(thrd)
{
    thrd = omp_get_thread_num(); // thread будет индивидуальной для каждой нити
}
```

Или например можно выставить более приоритетное количество потоков для секции через опцию `num_threads(num)`.

```
#pragma omp parallel private(thread) num_threads(10)
{ // будет запущен пул из 10 потоков
    thread = omp_get_thread_num();
}
```

Некоторые опции параллельной секции кода:

`if (условие)` — определяет условие выполнения параллельных потоков в последующем параллельном структурном блоке; если условие принимает значение истины, то потоки в последующем параллельном структурном блоке выполняются, в противном случае не выполняются.

`num_threads (целочисленное значение)` — задание количества нитей, которые будут выполнять параллельную область; по умолчанию выбирается последнее значение, установленное с помощью функции `omp_set_num_threads (int)`, или значение переменной `OMP_NUM_THREADS`.

`shared(список)` – задает список переменных, размещающихся в одной и той же области памяти для всех потоков.

`private(список)` – задает список переменных, локальных для каждого из параллельных потоков; в каждом из потоков эти переменные имеют собственные значения и относятся к различным областям памяти:

Локальным областям памяти каждого конкретного параллельного потока. Переменные требуется проинициализировать!

`default(shared | none)` – всем переменным параллельной области, которым явно не назначен класс `shared`; `none` означает, что всем переменным параллельной области должен быть явно назначен класс.

`firstprivate(список)` – задает список переменных, для которых порождается локальная копия для каждой нити, значения этих переменных инициализируются их значениями в нити-мастере.

`lastprivate(список)` – позволяет сохранить последние значения переменных в последней итерации параллельной области для использования вне этой области (только для циклов).

`reduction(оператор: список)` – определяется оператор для операции (+, −, *, / и т. п.), для которых будут вычисляться соответствующие частичные значения в параллельных потоках последующего параллельного структурного блока; кроме того, определяется список локальных переменных, в котором будут сохраняться соответствующие частичные значения; после завершения всех параллельных процессов частичные значения складываются (вычитаются, перемножаются и т. п.), и результат сохраняется в одноименной общей переменной. Так как стандарт не предусматривает фиксированную последовательность выполняемых действий, результат может варьироваться от запуска к запуску.

Выбор количества нитей в OpenMP

Для OpenMP существует способ задать количество ожидаемых в параллельной части программы нитей. Это возможно сделать с помощью переменной среды `OMP_NUM_THREADS`:

```
>> export OMP_NUM_THREADS = n
```

При выставлении переменной среды программа, написанная без уточнения количества нитей через функции или опции, будет порождать пул потоков из указанного количество потоков. Так же количество нитей можно уточнить с помощью функции `omp_set_num_threads(int)` уже непосредственно из кода. Можно уточнить количество нитей в пуле для концертной секции уже с помощью опции:

```
>> export OMP_NUM_THREADS = 5
#pragma omp parallel // будет создан пул из 5 нитей
{...}
...
omp_set_num_threads(10);
#pragma omp parallel num_threads(15) // будет создан пул из 15 нитей
{...}
#pragma omp parallel // будет создан пул из 10 нитей
{...}
```

Пример, демонстрирующий работу опции `reduction`.

```
#include <stdio.h>
int main()
{
    int n = 0;
    #pragma omp parallel num_threads (128) reduction (+: n)
    {
        n++;
        printf( "Текущее значение n: %d\n" , n );
    }
    printf( "Текущее значение n: %d\n" , n );
    return 0;
}
```

Опция `reduction` выделяет приватные копии переменных `n`, далее в нитях происходят процессы, в результате которых в `n` появляются части целого. После завершения параллельных секций для всех копий `n` выполняется операция `+`, результат которой складывается в `n`, выделенную ранее на стеке.

В программе могут быть использованы одновременно несколько параллельных областей. Максимальное количество нитей, которые могут быть задействованы для выполнения всей программы может быть задан с помощью переменной среды:

```
export OMP_THREAD_LIMIT=16
```

Позволить/запретить вложенные параллельные регионы можно с помощью переменной среды:

```
export OMP_NESTED=true
```

```
export OMP_NESTED=false
```


Кроме того OpenMP имеет режим, при котором количество создаваемых нитей при входе в параллельную область может меняться динамически. Сделать это можно с помощью функции:

```
void omp_set_dynamic(int dynamic_threads);
```

Узнать значение переменной можно:

```
int omp_get_dynamic(void);
```

Так же переменную можно поменять извне:

```
export OMP_DYNAMIC=true
```


Каждая нить представляет собой независимо выполняющийся поток управления со своим счетчиком команд, регистровым контекстом и стеком. Переменная `stack-size-var` задает размер стека, можно изменить её значение через:

```
export OMP_STACKSIZE=200000B  
export OMP_STACKSIZE="3000 k "  
export OMP_STACKSIZE=10M  
export OMP_STACKSIZE="1G"
```

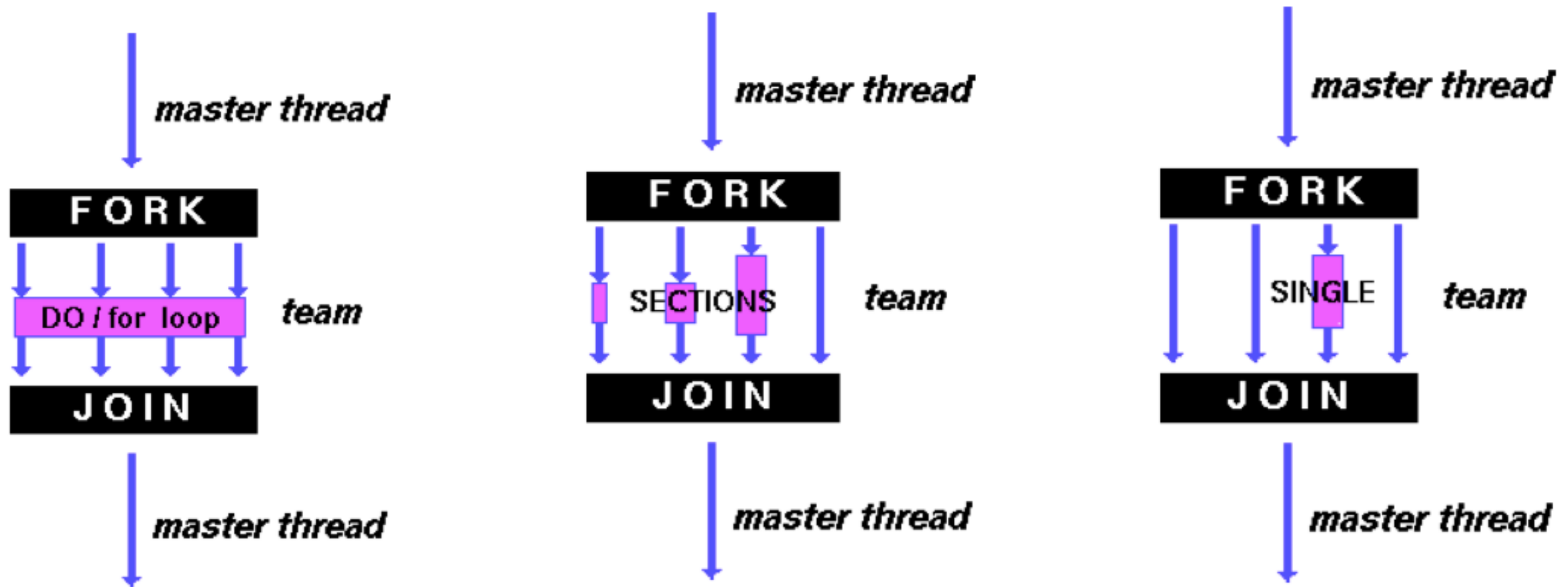
Нити можно привязать к логическим ядрам:

```
export OMP_PROC_BIND=true  
export OMP_PROC_BIND=false
```

Также можно выбрать желаемое поведение нитей во время ожидания:

```
export OMP_WAIT_POLICY=active  
export OMP_WAIT_POLICY=passive
```

Способы разделения работы между нитями



```
#pragma omp for
for (i=0;i<N;i++) {
  // code
}
```

```
#pragma omp sections
{
  #pragma omp section
  // code 1
  #pragma omp section
  // code 2
}
```

```
#pragma omp single {
  // code
}
```

OpenMP позволяет выполнять код только главной нитью, с помощью директивы master:

```
int main() {
    int i;
    #pragma omp parallel private(i)
    {
        #pragma omp for
        for (i=0; i<10; i++)
            printf("1 %d %d\n", omp_get_thread_num(), i);

        #pragma omp master
        printf("I'm Master!\n");

        #pragma omp for
        for (i=0; i<10; i++)
            printf("2 %d %d\n", omp_get_thread_num(), i);
    }
    return 0;
}
```

Директива `single` служит для того, чтобы выполнить некий блок в параллельной области только один раз. Часто используется для выводов, чтобы только одна нить печатала информацию.

```
int main(){
    int i;
    #pragma omp parallel private(i)
    {
        #pragma omp for
        for (i=0 ; i < 10 ; i++){
            printf("1 %d \n",i);
        }

        #pragma omp single
            printf("I'm thread %d!\n", omp_get_thread_num());

        #pragma omp for
        for (i=0 ; i < 10 ; i++){
            printf("2 %d \n",i);
        }
    }
    return 0;
}
```

Директива `critical`. Блок после директивы `critical` выполнится всеми нитями, но по очереди. Если блок уже исполняется любой нитью, то все остальные блокируются на входе и ждут завершения. После завершения работы, случайным образом из списка заблокированных выбирается новая. Таким образом в один момент блок будет исполняться только одной нитью.

```
int main() {  
  
    int n = 1;  
  
    #pragma omp parallel shared(n)  
    {  
        #pragma omp critical      {  
            n++;  
            printf("n = %d\n", n);  
        }  
    }  
}
```

Области видимости переменных

Область видимости переменных, объявленные вне параллельного блока, определяются параметрами директив:

- `private`
- `firstprivate`
- `shared`
- `default`
- `reduction`
- `threadprivate`
- `copyin`
- `lastprivate`

`private` — своя локальная переменная в каждом потоке. Возможно компилятор не обнулит локальную переменную помеченную как `private`, это может быть источником ошибок.

```
int num;  
#pragma omp parallel private(num)  
{  
    num = omp_get_thread_num();  
printf("%d\n", num);  
}
```

`firstprivate` — локальная переменная с инициализацией

```
int num = 5;  
#pragma omp parallel firstprivate(num)  
{  
    printf("%d\n", num);  
}
```


`default` — Задание области видимости не указанных явно переменных

```
int i,k,n = 2;
#pragma omp parallel shared(n) default(none)
private(i,k)
{
    i = omp_get_thread_num() / n;
    k = omp_get_thread_num() % n;
    printf( "%d %d %d\n", i, k, n );
}
```

`reduction` — Переменная для выполнения редукционной операции

```
int i,s = 0;
#pragma omp parallel for reduction(+:s)
{
    s += i;
}
printf( "Sum: %d\n", s );
```


threadprivate — Объявление глобальных переменных локальными для потоков

```
int x;
#pragma omp threadprivate(x)
int main() {
    ...
    #pragma omp parallel
    ...
}
```

copyin — Объявление глобальных переменных локальными для потоков с инициализацией

```
int x = 0;
#pragma omp threadprivate(x)
int main() {
    ...
    #pragma omp parallel copyin(x)
    ...
}
```

`shared` — Разделяемая (общая) переменная

```
int i,j;  
#pragma omp parallel for shared(j)  
{  
...  
}
```